

Raíces que siempre están ahí

Las raíces virtuales de un polinomio real, verificadas por máquina en Lean 4

Carles Marín*

Investigador independiente
karlesmarin@gmail.com

Junio 2026

Abstract

Un polinomio real de grado d puede tener menos de d raíces reales, pero siempre tiene exactamente d *raíces virtuales*: sustitutos continuos y semialgebraicos $\rho_{d,1}(p) \leq \dots \leq \rho_{d,d}(p)$ que coinciden con las raíces reales cuando estas existen y, si no, se sitúan donde el polinomio más se acerca a anularse. Las introdujeron González-Vega, Lombardi y Mahé [1] y son el portador natural del recuento de Budan–Fourier [2]. Reporto una formalización completa y **sorry-free** en Lean 4 sobre Mathlib de su *teoría estructural*: la construcción recursiva mediante un operador de elección $\mathcal{R}_d$ que minimiza $|p|$ en cada intervalo donde p' mantiene signo constante; el recuento fundamental, que hay exactamente d (`vroots_length`); que salen ordenadas y dentro del intervalo de encuadre (`vroots_isChain`, `vroots_subset_Icc`); que $\mathcal{R}_d$ devuelve una raíz real donde p tiene una (`R_eval_eq_zero_of_le_zero_le`); y el *entrelazado de Rolle*

$$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p),$$

que dice que cada raíz virtual de p' queda encajada entre dos raíces virtuales consecutivas de p (`vroots_interlacing`). Sobre esa estructura pruebo después el recuento exacto de Budan–Fourier mismo: el número de raíces virtuales en $(a, b]$ iguala la caída de variaciones de signo $V(a) - V(b)$ de la torre de derivadas (`card_vroots_Ioc_eq_fourierVar`, Sección 6)—la igualdad que el teorema clásico de Budan–Fourier solo acota para el recuento tembloroso de raíces *reales*. Esto funde la construcción de aquí con una segunda, recursiva de forma independiente, el recuento de signos `fourierVar` de la formalización compañera de Budan–Fourier [18]. Hasta donde sé—a partir de búsquedas en junio 2026 de Mathlib, la biblioteca `mathcomp-real-closed` de Coq, el AFP de Isabelle y HOL Light (Sección 9)—ni las raíces virtuales ni este recuento exacto a través de ellas se habían formalizado en ningún asistente de pruebas; este es su primer desarrollo. Todo teorema depende solo de los tres axiomas estándar `propext`, `Classical.choice`, `Quot.sound`.

Lo que encontrarás aquí. Una visita guiada breve, construida como una cadena de preguntas, una por sección (Figura 1): *cuántas* raíces tiene de verdad un polinomio, una vez que el recuento deja de temblar (§2); *cómo* construir las d mediante un operador de elección $\mathcal{R}_d$ (§3); *por qué* esa construcción se resiste a hacerse formal (§4); *cómo* se entrelazan, a la manera de Rolle (§5); y *cuántas* caen en un intervalo—el recuento exacto de Budan–Fourier (§6). El desarrollo son dos ficheros Lean, `VirtualRoots.lean` (la estructura) y `VirtualRootsCount.lean` (el recuento); los lemas que soportan el peso están en la Sección 8, los límites honestos en la Sección 9. Si solo lees dos cosas, lee el entrelazado (Teorema 1) y el recuento (Teorema 2).

*Formalizado con la asistencia de un sistema de IA (Claude, Anthropic); la matemática es clásica, y toda afirmación,

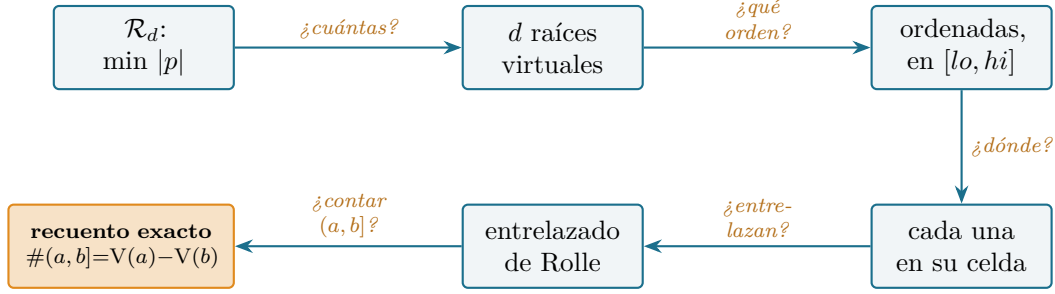


Figure 1: El mapa del lector. Del operador de elección $\mathcal{R}_d$ a un recuento certificado en un intervalo; cada flecha es la pregunta que responde su sección, y cada nodo es un bloque de Lean `sorry-free`. La cadena limpia—operador, d raíces, orden, localización por celda, entrelazado, recuento—es la arquitectura sobre la que se sostiene todo el desarrollo.

1 Un recuento que no tiembla

¿Cuántas raíces tiene un polinomio? La respuesta honesta tiembla. Un polinomio real de grado d tiene *entre* 0 y d raíces reales, y el número da bandazos al mover los coeficientes: $x^2 - t$ tiene dos raíces reales, luego una, luego ninguna a medida que t cae por debajo de cero, mientras su grado nunca cambia. Un recuento que salta cuando nada esencial lo ha hecho es un mal recuento. **Hay uno mejor: un polinomio de grado d tiene siempre exactamente d raíces virtuales**—números reales $\rho_{d,1}(p) \leq \dots \leq \rho_{d,d}(p)$ que coinciden con las raíces genuinas cuando existen y, si no, se sitúan donde el polinomio más se acerca a anularse. No saltan, no desaparecen y varían de forma continua con los coeficientes. Este paper verifica por máquina la teoría estructural de ese recuento mejor.

La imagen que conviene retener es la más simple no trivial. El polinomio $p = x^2 + 1$ no tiene raíz real, pero su gráfica baja a un único acercamiento máximo en $x = 0$. Las raíces virtuales ven ese acercamiento: p tiene la raíz virtual doble $\rho_{2,1} = \rho_{2,2} = 0$ (Figura 2). Donde habría una raíz real, hay una virtual; donde se esconden dos raíces conjugadas, las dos raíces virtuales colisionan en el fondo de la hondonada en vez de desaparecer. Nada se inventa—la construcción es explícita y semialgebraica—y nada se pierde: **el recuento siempre es d** . El hilo conductor del paper es ese contraste: las raíces reales son un recuento que tiembla y las virtuales son la familia estable de d elementos por debajo, y la estabilidad no se afirma sino que se *construye*, lo bastante concreta como para que un núcleo la verifique.

La pregunta es vieja, y también sus respuestas. Descartes (1637) acotó las raíces positivas por los cambios de signo de los coeficientes; Budan y Fourier (1807–1831) leyeron el mismo recuento en la torre de derivadas $p, p', p'', \dots$; Sturm (1835) lo hizo exacto, pero solo para raíces *distintas*. Cada una de estas reglas cuenta una cantidad que tiembla—las raíces reales—así que las de Budan y Fourier solo la *acotan*, con un excedente par que nadie supo nombrar. El arreglo llegó tarde: González-Vega, Lombardi y Mahé aislaron las raíces virtuales en 1998 [1], la familia estable de d elementos para la que esa cota se vuelve igualdad (Coste, Lajous, Lombardi y Roy [2]). Este paper verifica por máquina esa familia—su construcción, su recuento, su orden y el entrelazado de Rolle—y luego la igualdad misma (Sección 6): el excedente, resulta, no era más que el recuento mirando las raíces equivocadas. Llegué a él como llegué a Sturm [19] y a Budan–Fourier [18]: preguntando qué resultados clásicos de recuento de raíces le faltan a una biblioteca madura, y hallando las raíces

delimitación de alcance y limitación es responsabilidad del autor. La frontera de la contribución—en particular lo que *aún no* está formalizado—se expone sin adornos en la Sección 9, y es el núcleo de Lean, no el asistente, quien certifica las pruebas.

virtuales ausentes de todo asistente que pude buscar.

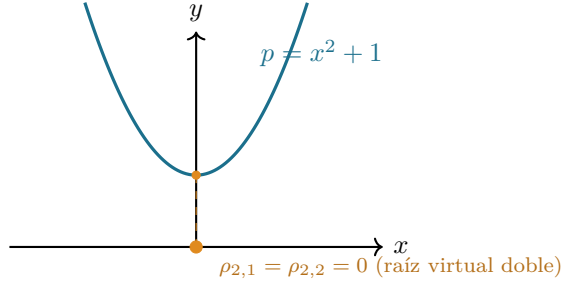


Figure 2: Raíces que siempre están ahí. El polinomio $x^2 + 1$ no tiene raíz real, pero sí una raíz virtual doble en $x = 0$, donde la gráfica más se acerca al eje. Un polinomio de grado 2 tiene dos raíces virtuales sea cual sea su discriminante; aquí coinciden. En Lean, sobre un intervalo de encuadre, la construcción devuelve la lista $[0, 0]$.

2 Las d raíces que siempre están ahí

La idea de que un polinomio carga d “raíces” reales canónicas con independencia de cuántas sean genuinas es de González-Vega–Lombardi–Mahé [1], enmarcada en la teoría de variación de signos cuyo tratamiento de libro es Basu–Pollack–Roy [11], Cap. 2. La definición es `vroots` y el recuento es `vroots_length`.

Entre dos puntos críticos consecutivos de p —dos raíces consecutivas de p' —la derivada mantiene signo constante, así que p es estrictamente monótona ahí y $|p|$ tiene un único mínimo. El punto donde se alcanza ese mínimo es una raíz virtual: la única raíz de p en la celda si p cambia de signo a través de ella, y si no, el extremo de la celda donde p es menor. Los puntos críticos de p son las raíces virtuales de p' , así que la construcción recurre en la derivada, igual que el teorema de Rolle [6] intercala las raíces de p y p' .

Por qué exactamente d . Un polinomio de grado d tiene una derivada de grado $d - 1$, luego—por inducción— $d - 1$ raíces virtuales de p' ; estas cortan la recta en d celdas, y cada celda aporta una raíz virtual de p . El recuento nunca degenera: es el grado, no el número de raíces reales. Este es el primer teorema, y el más limpio:

$$\# \{\text{raíces virtuales de } p\} = \deg p.$$

En Lean es `vroots_length`: la lista `vroots lo hi p` de raíces virtuales tiene longitud `p.natDegree`. Las raíces reales son a lo sumo d (y pueden ser muchas menos, o ninguna); **las virtuales son siempre exactamente d** . La diferencia entre ambos recuentos es precisamente el excedente par de Budan–Fourier, ahora realizado como colisiones de raíces virtuales fuera del eje.

La receta es limpia sobre el papel—*minimiza $|p|$ en cada celda p' -monótona*—pero “el punto donde $|p|$ es menor” es una elección, y “las celdas” las corta un objeto definido por la misma recursión. Ambas han de hacerse explícitas antes de que un núcleo las acepte.

3 Construírlas: el operador de elección $\mathcal{R}_d$

La construcción recursiva—cortar la recta por los puntos críticos, minimizar $|p|$ en cada trozo—es la definición de González-Vega–Lombardi–Mahé [1], ligada al recuento de variación de signos

por Coste [2]. Donde una prueba a mano dice “toma el punto donde $|p|$ es menor”, una formal debe producirlo: ese es el operador $\mathcal{R}$ (existencia `exists_isMinOn_abs_eval`), ensamblado por la recursión `vroots`.

Las celdas de p las cortan las raíces virtuales de p' junto con dos vallas exteriores. En vez de trabajar en todo $\mathbb{R}$ desde el principio, fijo un intervalo cerrado $[lo, hi]$ y construyo las raíces virtuales dentro de él; tomar lo, hi más allá de toda raíz de la torre de derivadas (una cota de Cauchy) recupera el objeto global, y ese último paso se comenta en la Sección 9. Trabajar en $[lo, hi]$ mantiene toda cantidad como número real genuino y deja a la recursión reutilizar un único operador de elección.

El operador de elección $\mathcal{R}_d$. En un intervalo cerrado $[a, b]$, la función continua $|p|$ alcanza un mínimo (el intervalo es compacto); llamamos $\mathcal{R}_d(a, b, p)$ a un minimizador. En Lean es $\mathcal{R}$, definido eligiendo un minimizador cuya existencia es `exists_isMinOn_abs_eval`. Dos hechos sobre él cargan toda la teoría. Primero, **cae en el intervalo**: `R_mem` dice $\mathcal{R}_d(a, b, p) \in [a, b]$. Segundo, **capta las raíces reales**: si $p(a) \leq 0 \leq p(b)$ (o al revés), el teorema del valor intermedio da una raíz, el mínimo de $|p|$ es entonces 0, y el minimizador es esa raíz—`R_eval_eq_zero_of_le_zero_le`. Así que en una celda donde p cambia de signo, $\mathcal{R}_d$ es la raíz genuina; donde no, es el extremo más cercano. Ese es el sentido preciso en que las raíces virtuales extienden a las reales.

El operador de elección $\mathcal{R}_d(a, b, p)$ (receta)

En un intervalo cerrado $[a, b]$, toma **cualquier minimizador de $|p|$** (existe, $[a, b]$ es compacto). Dos hechos bastan para toda la teoría. **Cae en el intervalo**, $\mathcal{R}_d(a, b, p) \in [a, b]$ (`R_mem`); este único hecho da el entrelazado. **Es una raíz real cuando p tiene una**: si p cambia de signo en $[a, b]$, el mínimo de $|p|$ es 0, luego $\mathcal{R}_d$ es una raíz genuina (`R_eval_eq_zero_of_le_zero_le`); así extienden las virtuales a las reales. La unicidad, donde p' mantiene signo, sale gratis de los ladrillos de monotonía de la sección siguiente—así que el minimizador *elegido* es de hecho el único.

La recursión. Las raíces virtuales se ensamblan listando los breakpoints

$$\text{bps} = lo :: (\text{raíces virtuales de } p') ++ [hi],$$

y aplicando $\mathcal{R}_d$ a cada par consecutivo: `vroots lo hi p = zipWith ( $\mathcal{R}_d p$ ) bps bps.tail`. Un polinomio de grado 0 no tiene raíces virtuales; en otro caso la recursión desciende a p' , terminando porque $\deg p' < \deg p$ (`natDegree_derivative_lt`). La definición bien fundada y el teorema de longitud son el contenido de `vroots` y `vroots_length` (Figura 3).

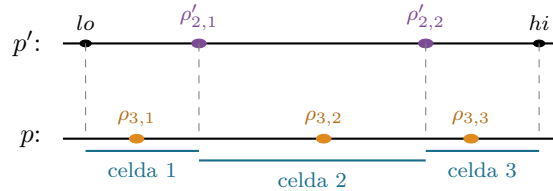


Figure 3: La recursión, para $\deg p = 3$. Las dos raíces virtuales de p' (ciruela) junto con las vallas lo, hi cortan $[lo, hi]$ en tres celdas; en cada celda p' mantiene signo constante, así que $\mathcal{R}_d$ devuelve una raíz virtual de p (ámbar). Cada $\rho_{3,j}$ cae en su propia celda—esto es lo que las hace salir ordenadas y entrelazadas con las $\rho'_{2,j}$.

4 Hacer formal la construcción

La construcción es clásica y el entrelazado tiene una prueba a mano de una línea—“ $\mathcal{R}_d$ cae en su celda, y las celdas comparten extremos”—así que lo que cuesta el trabajo no es matemática nueva sino la contabilidad que hace que una familia recursiva y definida por elección pase un núcleo, el análogo de la cirugía de listas que necesitó el desarrollo compañero de Budan–Fourier [18]. Entra en `vroots_chain_mem` (el invariante maestro) y `isChain_zipWith_R`.

Una elección no computable, usada con honestidad. $\mathcal{R}_d$ es un minimizador *elegido*, así que `R` es `noncomputable` y el desarrollo se apoya en `Classical.choice`. Es válido y estándar, pero significa que las raíces virtuales están especificadas, no calculadas: los teoremas son sobre *un* minimizador, y lo que los hace inequívocos es la monotonía. Los dos *ladrillos de monotonía* compartidos con el trabajo compañero de Budan–Fourier—`eval_strictMonoOn_of_derivative_pos` y su gemelo negativo—dicen que donde p' mantiene signo constante no nulo, p es estrictamente monótona, así que $|p|$ tiene un *único* pozo y el minimizador es único. El desarrollo formal solo necesita la existencia y los dos hechos `R_mem / R_eval_eq_zero`; la unicidad es lo que licencia la imagen informal.

Los breakpoints forman una cadena, y ese es el quid. Todo lo de aguas abajo descansa en un invariante: la lista aumentada de breakpoints `lo :: (vroots lo hi p') ++ [hi]` está *ordenada*, y toda raíz virtual cae en $[lo, hi]$. Las dos mitades son mutuamente dependientes—una lista está ordenada solo si sus miembros están en rango, y $\mathcal{R}_d$ cae en rango solo entre breakpoints ordenados—así que se prueban juntas, por una única inducción que baja la torre de derivadas (`vroots_chain_mem`). El orden es el motor: como $\mathcal{R}_d(\text{bps}[r], \text{bps}[r+1])$ cae en $[\text{bps}[r], \text{bps}[r+1]]$ y los breakpoints consecutivos están ordenados, las raíces virtuales consecutivas también lo están (`isChain_zipWith_R`). Es el teorema de Rolle en forma de lista, y es la parte que la máquina no me dejó esquivar.

Indexar, y las trampas pequeñas. Convertir “cada raíz virtual cae entre sus breakpoints” en un enunciado sobre `getElem` exigió un cuidado que una prueba a mano oculta. Reescribir un acceso indexado que carga prueba (`l[r]` arrastra una prueba $r < l.length$) no es un `rw` sin más; pasa por `simp` con los lemas de índice, apoyándose en la irrelevancia de pruebas para conciliar las cotas. Y un `rcases` destructivo sobre una ecuación $w = lo$ *sustituye lo* en silencio en esa rama—un arreglo de un carácter (`le_rfl` en vez de `le_refl lo`), pero del tipo que solo un núcleo detecta. No son profundas, pero son justo los gestos de la mano que un asistente de pruebas rechaza.

La cadena y la localización por índice, una vez probadas, son exactamente sobre lo que descansa el entrelazado.

5 El entrelazado de Rolle

Que las raíces de p y p' se intercalen es de Rolle, de 1690 [6]; que las raíces *virtuales* se intercalen del mismo modo—incluso cuando no hay raíces reales—es el refinamiento de González-Vega–Lombardi–Mahé [1] y el núcleo estructural del teorema de recuento de Coste [2]. Es `vroots_interlacing`, que descansa en la localización por índice `vroots_getElem_mem`.

La consecuencia principal es el entrelazado de las raíces virtuales de p con las de p' :

Teorema 1 (Entrelazado de Rolle de raíces virtuales; `vroots_interlacing`). *Para $lo \leq hi$ y cada índice r en rango,*

$$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p),$$

es decir, $(vroots\ lo\ hi\ p)[r] \leq (vroots\ lo\ hi\ (derivative\ p))[r] \leq (vroots\ lo\ hi\ p)[r+1]$.

La prueba son dos aplicaciones de la localización `vroots_getElem_mem`—cada raíz virtual de p cae entre dos breakpoints—junto con la identidad de que los breakpoints interiores *son* las raíces virtuales de p' . Es la sombra discreta del teorema de Rolle, y sitúa a la familia de raíces virtuales en el mismo plano que las familias entrelazadas que recorren la literatura de real-rootedness—las desigualdades de Newton, el polinomio de matching de Heilmann–Lieb—donde una sucesión y su derivada se intercalan. Que esto valga ahora para una familia definida incluso cuando no hay raíces reales es el punto.

De qué es el camino. El entrelazado es la mitad estructural de la historia de Budan–Fourier. La mitad aritmética—que el número de raíces virtuales en un intervalo semiabierto $(a, b]$ iguala la caída de variaciones de signo $V(a) - V(b)$ de la torre de derivadas [2]—es la igualdad que la desigualdad clásica de Budan–Fourier [18] solo acota. Funde esta construcción con una segunda, recursiva de forma independiente, y por eso era la secuela; es el objeto de la sección siguiente, ya probada.

6 El recuento exacto

El entrelazado era, en la primera versión de este trabajo, donde el paper se detenía: la teoría estructural estaba probada y el recuento aritmético se nombraba con honestidad como la secuela. Ya está probado, y esta sección lo reporta. Es la igualdad para la que era toda la construcción.

Teorema 2 (El recuento exacto de Budan–Fourier; `card_vroots_Ioc_eq_fourierVar`). *Supóngase que toda raíz de todo miembro de la torre de derivadas $p, p', p'', \dots$ cae estrictamente dentro de $[lo, hi]$. Entonces para $a \leq b$ en $[lo, hi]$,*

$$\#\{\rho \text{ raíz virtual de } p : a < \rho \leq b\} = V(a) - V(b),$$

donde $V = \text{fourierVar}$ es el recuento de variaciones de signo de Budan–Fourier de la torre en un punto.

En Lean el enunciado es, verbatim (los binders de tipo implícitos $\{lo\ hi : \mathbb{R}\}$, $\{p : \mathbb{R}[X]\}$ omitidos):

```
theorem card_vroots_Ioc_eq_fourierVar (hp : p ≠ 0)
  (hbr : Brackets lo hi p) (ha : a ∈ Ico lo hi) (hb : b ∈ Ico lo hi) (hab :
a ≤ b) :
  ((vroots lo hi p : Multiset ℝ).filter (fun r => a < r ∧ r ≤ b)).card
  = fourierVar p a - fourierVar p b
```

El teorema clásico de Budan–Fourier [18] acota el número de raíces *reales* en $(a, b]$ por $V(a) - V(b)$, con un excedente par. Reemplaza las raíces reales por las virtuales y la desigualdad se vuelve igualdad—el excedente nunca fue un defecto del recuento, solo de *cuáles* raíces se contaban. El recuento tembloroso de la Sección 1 es lo que hacía de Budan–Fourier una desigualdad; el recuento estable lo vuelve exacto (Figura 4).

Por qué es un teorema y no una línea. Funde las dos recursiones de las que está hecho el desarrollo. Tanto V como `vroots` bajan la misma torre $p \rightarrow p' - V$ pelando el signo principal, `vroots` por el entrelazado de Rolle—pero son objetos *a priori* distintos, y el teorema es que marchan al unísono. La prueba pasa por la reformulación

$$V_p(x) = \#\{\text{raíces virtuales de } p \text{ estrictamente por encima de } x\}$$

(`fourierVar_eq_card_vroots_gt`, por inducción que baja la torre), de la que el recuento en $(a, b]$ sale por resta; el teorema del intervalo es entonces una partición de multiconjunto. El paso inductivo

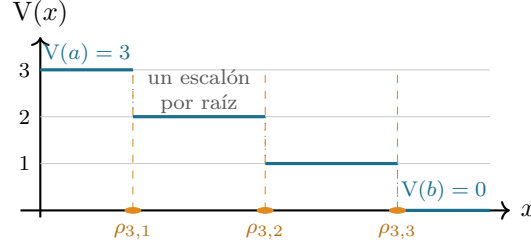


Figure 4: El recuento exacto como escalera. Para $p = x^3 - x$ el recuento de Budan–Fourier $V(x)$ es plano entre raíces virtuales y baja exactamente uno en cada una; el número de raíces virtuales en $(a, b]$ es el descenso total $V(a) - V(b)$. Aquí $V(a) - V(b) = 3$ para cualquier $a < -1$ y $b > 1$, coincidiendo con las tres raíces virtuales $\rho_{3,1} < \rho_{3,2} < \rho_{3,3}$ (todas reales para este p). Una colisión de grado dos, como en la Figura 2, aparecería como un único escalón de altura dos—el recuento sigue exacto, leyendo ahora una raíz virtual que el recuento de raíces reales no ve.

es un único lema por escalón (`count_step`): de p' a p , el recuento de raíces virtuales por encima de x sube exactamente el incremento de Fourier $V_p(x) - V_{p'}(x)$, y ese incremento es 0 o 1.

Dónde se encuentran los dos bits. Localiza x en su celda $[\text{bps}[i], \text{bps}[i + 1]]$ de la partición de breakpoints de p' (Figura 3). El entrelazado ya obliga a cada celda estrictamente por encima de x a aportar una raíz virtual de p y a cada celda por debajo a no aportar ninguna, así que todo el incremento es si la única raíz virtual v_i en la propia celda de x cae por encima de x (`count_diff_eq_cell`). En esa celda p' no tiene raíz—una raíz de p' sería una raíz virtual de p' , es decir un breakpoint, y ninguno cae estrictamente entre dos breakpoints consecutivos (Prop. 2.2 de Coste, aquí `root_mem_vroots`)—así que p es estrictamente monótona a través de ella, y su minimizador de $|p|$, v_i , cae por encima de x exactamente cuando $p(x)$ tiene el signo que aún no ha “doblado la esquina” hacia la raíz (`lt_R_iff_eval_neg` y su gemelo).

El único paso analítico. En el lado de Fourier el incremento es 1 exactamente cuando $\text{sign } p(x)$ discrepa del primer signo superviviente d de la torre $p', p'', \dots$ en x . El puente entre los dos lados es una identidad:

$$d = \text{el signo de } p' \text{ en la celda de } x.$$

La derivada de orden más bajo que no se anula en x fija p' justo a la derecha de x , y en la celda sin raíces ese signo es constante; así que d es la dirección de monotonía de la celda. (En Lean es `fourier_head_right`, reutilizando `signs_right_eq_Rseq` del trabajo compañero [18]: el patrón de signos justo a la derecha de x es el patrón superviviente principal en x .) Esta identidad es la que elimina la contabilidad de multiplicidades que vuelve delicada la prueba clásica: una vez fijado d a la celda, “ v_i por encima de x ” y “una nueva variación de Fourier” son el *mismo* suceso, y las dos recursiones coinciden término a término. Sumar los escalones da el Teorema 2 (Figura 4).

Qué necesita y qué no. La hipótesis de encuadre—toda raíz de la torre estrictamente dentro de $[lo, hi]$ —es justo lo que hace limpio el recuento: $V(lo)$ es el grado, $V(hi) = 0$, y toda raíz virtual es interior. Es la misma disciplina de intervalo fijo que la construcción (Sección 3); levantarla a todo $\mathbb{R}$ por una cota de Cauchy es el mismo paso diferido (Sección 9). No se calcula ninguna raíz ni se construye ningún aislador—el teorema certifica el recuento, no un procedimiento.

7 Qué habilita una teoría certificada de raíces virtuales

Una familia de raíces virtuales verificada por máquina es el término medio que falta entre dos cosas que Mathlib ya tiene y una que no tenía. Tiene la regla de Descartes (el recuento de signos de coeficientes) y, en el trabajo compañero, la *desigualdad* de Budan–Fourier; no tenía el objeto que vuelve esa desigualdad una igualdad. Las raíces virtuales son ese objeto, y la Sección 6 cierra la igualdad. Aguas abajo, son la entrada correcta para un procedimiento certificado de *aislamiento* de raíces reales en la línea de Vincent, Collins y Akritas [10, 9, 8] y la teoría de variación de signos de Basu–Pollack–Roy [11]—el recuento es ahora exacto, así que un método de subdivisión que lee $V(a) - V(b)$ en cada mitad tiene, en principio, una garantía verificada sobre la que construir. Y como la familia está definida para todo polinomio, con raíces reales o sin ellas, conecta los métodos de intervalo con el álgebra del discriminante: un par de raíces virtuales colisiona exactamente cuando dos raíces reales están a punto de volverse complejas, que es de donde sale el excedente par de Budan–Fourier (Figura 5). La teoría estructural formalizada aquí es el cimiento sobre el que se sostiene todo eso.

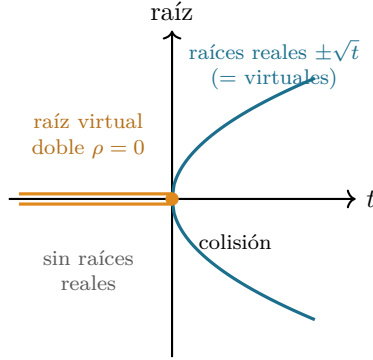


Figure 5: Raíces que siempre están ahí, al mover un parámetro. Para $p_t = x^2 - t$ las dos raíces reales $\pm\sqrt{t}$ (teal) existen solo para $t \geq 0$ y desaparecen al caer t por debajo de 0; las dos raíces *virtuales* (ámbar) persisten para *todo* t , coincidiendo con las reales cuando $t \geq 0$ y colisionando en la raíz virtual doble $\rho = 0$ para $t < 0$. El recuento de raíces reales salta $2 \rightarrow 0$ en $t = 0$; el de virtuales se queda en 2. La colisión en $t = 0$ es exactamente donde se forma un par conjugado—el origen del excedente par de Budan–Fourier. (Otra vista del mismo fenómeno que la Figura 2, ahora en movimiento.)

8 Las piezas

La Tabla 1 reúne los resultados que soportan el peso, todos `sorry-free` en `VirtualRoots.lean`; la Tabla 2 mide el desarrollo. La dependencia es corta: los ladrillos de monotonía dan la existencia y las dos propiedades de $\mathcal{R}_d$; la inducción combinada `vroots_chain_mem` prueba orden y rango a la vez; y la localización por índice da el entrelazado.

Nombre Lean	Enunciado	Estado
<code>R_mem</code>	$\mathcal{R}_d(a, b, p) \in [a, b]$	probado
<code>R_eval_eq_zero_of_le_zero_le</code>	$p(a) \leq 0 \leq p(b) \Rightarrow p(\mathcal{R}_d(a, b, p)) = 0$	probado
<code>vroots_length</code>	$\#\{\text{raíces virtuales de } p\} = \deg p$	probado
<code>vroots_isChain</code>	$lo :: (\text{vroots } p) ++ [hi]$ ordenada	probado
<code>vroots_subset_Icc</code>	toda raíz virtual cae en $[lo, hi]$	probado
<code>vroots_getElem_mem</code>	$\text{bps}[r] \leq \rho_{d,r}(p) \leq \text{bps}[r+1]$	probado
<code>vroots_interlacing</code>	$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p)$	probado
<code>fourierVar_eq_card_vroots_gt</code>	$V_p(x) = \#\{\text{raíces virtuales } > x\}$	probado
<code>count_step</code>	puente por escalón $V_p - V_{p'} = \text{incremento de celda}$	probado
<code>card_vroots_Ioc_eq_fourierVar</code>	$\# \text{ en } (a, b] = V(a) - V(b)$	probado

Table 1: La teoría de las raíces virtuales, toda `sorry-free`: los resultados estructurales en `VirtualRoots.lean` (arriba) y el recuento exacto en `VirtualRootsCount.lean` (abajo), Lean 4 / Mathlib. `#print axioms` sobre `vroots_interlacing` y sobre `card_vroots_Ioc_eq_fourierVar` reporta solo `propext`, `Classical.choice`, `Quot.sound`.

Cantidad	Valor
Líneas: <code>VirtualRoots.lean</code> / <code>VirtualRootsCount.lean</code>	378 / 944
Teoremas + defs: estructural / recuento	16+2 / 33
Módulos de Mathlib usados (ambos ficheros, distintos)	12
<code>sorry</code> / <code>admit</code> / <code>native_decide</code>	0
Axiomas fuera del núcleo de Lean	0 (solo <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>)
Compilación, Mathlib precompilado (<code>VirtualRoots</code> / recuento)	$\approx 8\text{s} / 9\text{s}$
Toolchain	Lean 4 (godsil pin v4.30) sobre Mathlib

Table 2: El desarrollo, medido. Build: `lake env lean VirtualRoots.lean` y `VirtualRootsCount.lean`, exit 0; el tiempo de compilación es para verificar cada fichero contra un Mathlib precompilado. La huella es deliberadamente ligera—nada de álgebra pesada, porque la construcción es elemental. Los doce módulos de Mathlib son `Analysis.Calculus.Deriv.Polynomial`, `Deriv.MeanValue`, `Topology.Algebra.Polynomial`, `Topology.Order.Compact`, `Topology.Order.IntermediateValue`, `Algebra.Polynomial.Derivative`, `Algebra.Polynomial.Roots`, `Algebra.Polynomial.FieldDivision`, `Data.List.Dstutter`, `Data.Sign.Basic`, `Data.Real.Basic` y `Algebra.Ring.Parity`.

9 La frontera de la contribución

Qué está probado. La teoría estructural de las raíces virtuales $\mathcal{R}_d$, `sorry-free`: existencia y recuento ($= \deg p$), que están ordenadas y caen en el intervalo de encuadre, que $\mathcal{R}_d$ devuelve una raíz real donde p cambia de signo, y el entrelazado de Rolle (`VirtualRoots.lean`); y, construido sobre ello, el recuento exacto de Budan–Fourier $\#\{(a, b]\} = V(a) - V(b)$ (`VirtualRootsCount.lean`, Sección 6). Son los resultados de González-Vega–Lombardi–Mahé [1] y Coste [2] en sus partes estructural y de recuento. El recuento se prueba sobre un intervalo de encuadre $[lo, hi]$ que contiene estrictamente toda raíz de la torre.

Qué no está probado. Dos cosas, ambas genuinamente más allá de los ficheros presentes. *La continuidad*: que las raíces virtuales varían de forma continua con los coeficientes—mitad de por qué las introdujeron González-Vega–Lombardi–Mahé [1]—no se toca (Sección 10). *El objeto global*: la construcción y el recuento son sobre un intervalo de encuadre fijo $[lo, hi]$; recuperar el objeto en

todo $\mathbb{R}$ tomando *lo, hi* más allá de toda raíz de la torre de derivadas (una cota de Cauchy, disponible en Mathlib como `Polynomial.cauchyBound`) es rutina pero no se realiza aquí.

Novedad. Hasta donde sé, ni las raíces virtuales ni el recuento exacto de Budan–Fourier a través de ellas se habían formalizado en ningún asistente de pruebas antes. La afirmación se apoya en búsquedas en junio 2026 de: Mathlib (sin raíces virtuales; tiene Descartes vía `Polynomial.signVariations`); la biblioteca `mathcomp-real-closed` de Coq (raíces reales y la codificación de Thom, no raíces virtuales); el AFP de Isabelle, cuya entrada `Budan_Fourier` formaliza la desigualdad con multiplicidad pero no el objeto raíz virtual ni la igualdad; y HOL Light (Sturm y Descartes, no raíces virtuales). Hago la afirmación al nivel del objeto nombrado; no puedo excluir un desarrollo inédito o con otro nombre, y agradecería una indicación.

Autoría. La matemática es clásica, debida a los autores citados. La formalización se realizó en interacción estrecha con un asistente de IA (Claude, Anthropic): la elección del objetivo, la arquitectura general de la prueba, los enunciados y las afirmaciones de alcance y novedad son mías; el asistente redactó e iteró buena parte del Lean bajo esa dirección. La división no afecta a la validez—una prueba se acepta solo cuando el núcleo de Lean la type-checkea, cosa que hace, con el perfil de axiomas reportado arriba—pero se enuncia aquí con claridad por atribución científica.

10 Qué queda abierto

Tres hechos marcan hacia dónde apunta el trabajo; el primero ya está resuelto y nombra su propia pregunta pendiente, los otros dos siguen abiertos.

1. *Dos definiciones, un objeto—desde cuál formalizar.* Las raíces virtuales tienen dos definiciones clásicas: la construcción por el minimizador $\mathcal{R}_d$ de González-Vega–Lombardi–Mahé [1], usada aquí, y los valores donde salta `fourierVar`, la definición de Coste et al. [2]. Las dos coinciden, y esa coincidencia es el teorema de Coste—no algo que este paper abra. Lo que este paper hace es formalizar la construcción y *puentearla* a `fourierVar` para el recuento (Sección 6). La definición por saltos haría el recuento casi por definición pero el entrelazado más difícil; así que lo genuinamente abierto no es la matemática sino la elección *en Lean*—¿es más barato, en un asistente de pruebas, re-derivar la equivalencia de Coste desde el lado de los saltos que el puente tomado aquí? El entrelazado fue gratis en el lado de la construcción, el recuento lo sería en el de los saltos; ninguna definición da ambas a la vez.
2. *La continuidad, la otra mitad de su razón de ser.* Las raíces virtuales varían de forma *continua* con los coeficientes—de hecho González-Vega–Lombardi–Mahé prueban que las $\rho_{d,j}$ son funciones semialgebraicas continuas [1], que es la mitad de por qué las introdujeron. La matemática es suya y está resuelta; lo abierto es solo su formalización, no tocada aquí. Una prueba en Lean tendría que hacer que $\mathcal{R}_d$, un minimizador elegido, dependa de p con continuidad; los ladrillos de monotonía que fijan el minimizador son el punto de partida natural, pero el enunciado es genuinamente analítico y va más allá de la contabilidad algebraica de este paper.
3. *Un entrelazado, tres caras.* El entrelazado de Rolle sitúa a las raíces virtuales junto a las familias que recorren la literatura de real-rootedness: las raíces de un polinomio y su derivada (Rolle clásico [6]), las desigualdades de Newton [20] y el polinomio de matching de Heilmann–Lieb. Las *cadenas de Sturm* abstractas de Eisermann [12] ya axiomatizan una de esas estructuras de entrelazado. ¿Hay una única noción Lean de “familia entrelazada” que subsuma todas estas—y dónde se sitúa dentro de ella la familia de raíces virtuales, definida incluso cuando no hay raíces reales? Esa es la cuenta que más me gustaría llevarme.

Reproducir

```
git clone https://github.com/karlesmarin/godsil-gutman-lean
cd godsil-gutman-lean && lake exe cache get
lake build BudanFourier          # el toolkit de variacion de signos importado
lake env lean VirtualRoots.lean  # la teoria estructural
lake env lean VirtualRootsCount.lean # el recuento exacto
```

Lean 4 (pin de toolchain godsil v4.30) sobre Mathlib. Los teoremas estructurales están en `VirtualRoots.lean` y el recuento exacto `card_vroots_Ioc_eq_fourierVar` en `VirtualRootsCount.lean`; sus soportes son la Tabla 1. Ambos ficheros son `sorry-free`, y `#print axioms` sobre `vroots_interlacing` y sobre `card_vroots_Ioc_eq_fourierVar` reporta solo `propext`, `Classical.choice`, `Quot.sound`. Los signos de cada figura ($x^2 + 1$, $x^3 - x$, $x^2 - t$) se re-derivan en aritmética exacta antes de dibujarla.

References

- [1] L. González-Vega, H. Lombardi, L. Mahé, Virtual roots of real polynomials, *J. Pure Appl. Algebra* **124** (1998), núm. 1–3, 147–166.
- [2] M. Coste, T. Lajous-Loeza, H. Lombardi, M.-F. Roy, Generalized Budan–Fourier theorem and virtual roots, *J. Complexity* **21** (2005), núm. 4, 479–486.
- [3] R. Descartes, *La Géométrie* (1637); la regla de los signos acota el número de raíces positivas por el número de cambios de signo de los coeficientes.
- [4] F. D. Budan de Boislaurent, *Nouvelle méthode pour la résolution des équations numériques*, París (1807; 2.^a ed. 1822).
- [5] J. Fourier, *Analyse des équations déterminées*, Firmin Didot, París (póstumo, 1831).
- [6] M. Rolle, *Traité d’algèbre* (1690); entre dos raíces reales consecutivas de una función derivable hay una raíz de su derivada.
- [7] C. Sturm, Mémoire sur la résolution des équations numériques, *Mémoires présentés par divers savants à l’Académie Royale des Sciences* **6** (1835) 271–318.
- [8] A. G. Akritas, Reflections on a pair of theorems by Budan and Fourier, *Mathematics Magazine* **55** (1982) 292–298.
- [9] G. E. Collins, A. G. Akritas, Polynomial real root isolation using Descartes’ rule of signs, en *Proc. SYMSAC ’76*, ACM, 1976, 272–275.
- [10] A. Alesina, M. Galuzzi, A new proof of Vincent’s theorem, *L’Enseignement Mathématique* **44** (1998) 219–256.
- [11] S. Basu, R. Pollack, M.-F. Roy, *Algorithms in Real Algebraic Geometry*, 2.^a ed., Algorithms and Computation in Mathematics **10**, Springer, 2006 (teoría de variación de signos, Cap. 2, y raíces virtuales).
- [12] M. Eisermann, The fundamental theorem of algebra made effective: an elementary real-algebraic proof via Sturm chains, *Amer. Math. Monthly* **119** (2012), núm. 9, 715–752 (“cadenas de Sturm” abstractas y el índice de Cauchy).

- [13] W. Li, L. C. Paulson, Counting polynomial roots in Isabelle/HOL: a formal proof of the Budan–Fourier theorem, en *Proc. CPP 2019*, ACM, 2019, 52–64; entrada del Archive of Formal Proofs “The Budan–Fourier Theorem” (W. Li, 2018), https://isa-afp.org/entries/Budan_Fourier.html (formaliza la desigualdad con multiplicidad, no el objeto raíz virtual).
- [14] C. Cohen, Construction of real algebraic numbers in Coq, en *Interactive Theorem Proving (ITP 2012)*, LNCS **7406**, Springer, 2012, 67–82 (base de `mathcomp-real-closed`: raíces reales y la codificación de Thom, no raíces virtuales).
- [15] M. Eberl, Sturm’s Theorem, *Archive of Formal Proofs* (2014), https://isa-afp.org/entries/Sturm_Sequences.html.
- [16] The mathlib Community, The Lean mathematical library, en *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.
- [17] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, en *Automated Deduction (CADE 28)*, LNCS **12699**, Springer, 2021, 625–635.
- [18] C. Marín, *Contar con la torre de derivadas: el teorema de Budan–Fourier, verificado por máquina en Lean 4* (2026, formalización compañera; `BudanFourier.lean`).
- [19] C. Marín, *The Staircase of Signs: Sturm’s Root-Counting Theorem, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20707348.
- [20] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton’s Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20693064.